INDEX

ANITS

S.No.	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
01	01/07	Introduction & basic HTML	CO1	10	
02	08/07	Design HTML using audio	CO1	10	
03	15/07	Design HTML using Video	CO1	10	
04	22/07	Program using JS (arrays)	CO1	10	
05	29/07	Develop dynamic webpage	CO1	10	
06	05/08	Implement modules in nodejs	CO1	10	
07	12/08	Mini application using express & nodejs	CO1	10	
08	19/08	Code tantra (static-webpage)	CO1	10	
09	2/09	Code tantra (Portfolio)	CO1	10	
10	9/09	Code tantra (Simple Calculator)	CO1	9	
11	16/09	Create a function & class	CO2	10	
12	23/09	Implement http methods	CO2	10	
13	30/9	Develop CRUD operation	CO3	10	
				10	

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B-Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712006222, Website: www.anits.edu.in, email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026-2029	**Semester:**	3-1
Name of the Student:	M. Mani Sankar	**Roll Number:**	A24126552159
Name of the Lab:	Full Stack Lab	**Timings:**	9:40 to 12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4 Marks)	Results (6 Marks)	Viva-Voce (2 Marks)			
1/7						15		
8/7						14		
15/7						15		
22/7						15		
29/7						15		
05/8						14		
12/8		4	4	5	2	15		
19/8		4	4	5	2	15		
2/9		4	4	5	2	15		
9/9		4	4	4	1	13		
16/9		4	4	4	2	14		
23/9		4	4	4	2	14		
30/9		4	2	4	2	13		

Name of the Faculty: Ch. Pratti

Faculty Signature: Chatti

Introduction to HTML and CSS:

HTML (hypertext markup Language) is the standard markup language used to create and structure content on webpages. It defines the basic structure of webpage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides structure and semantic organization of a webpage.

CSS (Cascading style sheets) is a stylesheet language used to control the presentation & visual appearance of HTML documents. It is used to define properties such as colors, fonts, background, spacing, borders, alignment, dimensions and layouts. CSS also supports responsive design, allowing webpages to adapt to different screen sized and devices CSS can be applied using inline, internal or external

* HTML and CSS work together to develop webpages, where HTML provides the structure and content, while CSS provides styling. The combination of both forms the foundation of modern web development and enables the creation of organized, attractive and user-friendly web interfaces

Difference between MERN stack and Full Stack:-

MERN stack is a javascript based technology stack used for developing modern web applications MERN stands for MongoDB, express.js, react.js, node.js, mongoDB is used for dabase management, express.js handles the backend framework and API's react.js is used to develop frontend user interface and node.js provides server side runtime environment.

* Full Stack development refers to development of both frontend and backend of a web applications along with databases integration. A full stack developer works with technologies.

HTML Elements:-

1) HTML tags :- Define the structure and elements of web page
2) Headings :- Used to display titles and heading using <h1> to <h6>
3) Paragraphs :- Used to display text using <p> tag
4) Links :- Used to navigate between webpages using <a> tag
5) Images :- Used to display images using <image> tag
6) Lists :- Used to create ordered & unordered lists using <ol>, <ul>,
7) Tables :- Used to organize data is rows and columns
8) Forms :- Used to collect user input using elements such as text boxes, buttons and checkboxes
9) Div and Span :- Used to group and organize webpage content
10) Semantic Elements :- Elements such as <header>, <nav>, <section>, <article>, <footer> provides meaningful webpage structure

CSS Components:-

1) Selectors - Select HTML elements to apply styles
2) Properties - Define the visual characteristics of element
3) Values - Specify the setting of css property
4) Colors - Define text, background, borders, colors
5) Fonts - Control font family, size, style and weight
6) Background - Set background colors and images
7) Borders - Define the border style, width and radius
8) Margin - Control space outside an element
9) Padding - Control the space inside an element
10) Layout - Control arrangement of elements using flex box, grip and positioning

HTML Vs HTML5:-

HTML	HTML 5
1) Older Version of HTML	1) Latest major Version HTML
2) Limited multimedia Support	2) Supports audio and Video
3) Uses external plugins for multimedia	3) No external plugs needed
4) Has fewer semantic tags	4) Provides semantic tags
5) No <canvas> element	5) Supports <canvas>
6) Limited web storage	6) Supports local storage.

CSS Vs CSS3

CSS	CSS 3
1) Earlier Version of CSS	1) Advanced Version
2) Basic styling Features	2) Provides advanced styling features
3) Limited Animations	3) Supports animations and transitions
4) No flexible layout System	4) Supports flexbox and grid

Design a static webpage using HTML and CSS

Pomodoro timer - Using HTML, CSS

```
<!DOCTYPE html>
<html>
<head>
    <title> Pomodoro Timer </title>
    <style>
        body {
            font-family : Arial;
            background : #f5f5f5;
            text-Align : Center;
            Padding : 50px;
        }
        timer {
            width : 300px;
            margin : auto;
            Padding : 30px;
            background : white;
            border-radius : 15px;
            box-shadow : 0 0 10px grey;
        }
        h1 {
            color : #e74c3c;
        }
        .time {
            font-size : 50px;
            margin : 20px;
        }
```

```
button{
        Padding: 10px 20px;
        margin: 5px;
        border: none;
        border-radius: 5px;
        background: #e74c3c;
        color: white;
    }
</style>
</head>
<body>
    <div class = "timer">
    <h1> Pomodoro timer </h1>
    <div class = "time"> 25:00 <div>
    <button> start </button>
    <button> Reset </button>
    </div>
</body>
</html>
```

01 Design an HTML Code using audio Tage

```
<!DOCTYPE html>
<html>
  <head>
    <title> Audio </title>
  </head>
  <body>
    <h2> my favourite song </h2>
    <audio controls>
      <source src = "song.mp3" type = "audio/mpeg">
      Your Browser doesn't support audio
      tag
    </audio>
  </body>
</html>
```

O/P

Displays an audio player with play, pause and Volume Controls



Steps

Create a new html file

Add the basic html structure

Add heading using <h2> tag

Insert the <audio> tag with control attribute

Specify the audio file using <source> tag

Save & open the Html file in a web browser.

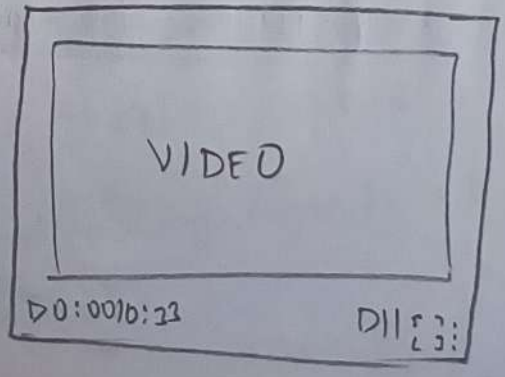
02 Design a HTML code using Video tags

```
<!DOCTYPE html>
<html>
  <head>
    <title> Video example </title>
  </head>
  <body>
    <h2> my Video </h2>
    <Video width = "400" Controls >
      <source src = "Video.mp4" type = "Video/mp4">
    </Video>
  </body>
</html>
```

Output

Displays an audio player with play, Pause and Volume Controls

- My Favourite ~~song~~ Video



Steps:-

Create a new html file

Add the basic Html structure

Add a heading using <h2> tag

insert the <video> tag with control attributes

Specify the video file using <source> tag

Save the file & open it in web browser

03 Write a program using Javascript (Arrays, Functions)

```
Let numbers = [12, 13, 14, 15, 16, 19, 24, 34, 67, 90, 99]

Function display(arr) {
    Console.log("Array Elements:");
    for (let i=0; i<arr.length; i++) {
        Console.log(arr[i]);
    }
}

Function Findsum(arr) {
    let sum=0;
    for (let i=0; i<arr.length; i++) {
        Sum += arr[i]
    }
    return sum;
}
```

O/P

```
12
13
14
15
16
19
24
34
67
90
99
Sum: 403
```

Experiment-04

04) Develop a dynamic webpage using java script

Program:-

```
<!DOCTYPE html>
<html>
<head>
    <title> Dynamic Webpage </title>
    <style>
        body{
            text-align: center;
            font-family: Arial;
            margin-top: 100px;
        }
        button{
            Padding 10px 20px;
            font-size: 16px;
            Cursor: pointer;
        }
    </style>
</head>
<body>
    <h1 id = "heading"> Welcome to My Webpage</h1>
    <p id = "message"> click the button to change
            this webpage </p>
    <button on click = "changePage()"> Click Me
                                    </button>
```

```
<script>
    function changePage(){
        document.getElementById("heading").
        innerHTML = "Welcome to Javascript!";
        document.getElementById("message").
        innerHTML = "The Webpage has been
                    Changed dynamically.";
        document.body.style.backgroundcolor
                    = "lightblue";
</script>
</body>
</html>
```

Output:-

Welcome to My Webpage

Click the button to change this webpage

[Click Me]

* When the user clicks "click Me", the javascript function changePage() is called, then the heading dynamically changes and background also changes

After clicking the button:

Welcome to Javascript!

The Webpage has been changed dynamically

[Click Me]

We will load them by module names:

```
// CommonJS style
Const fs = require('fs');
//Es modules style
import fs from 'fs';
```

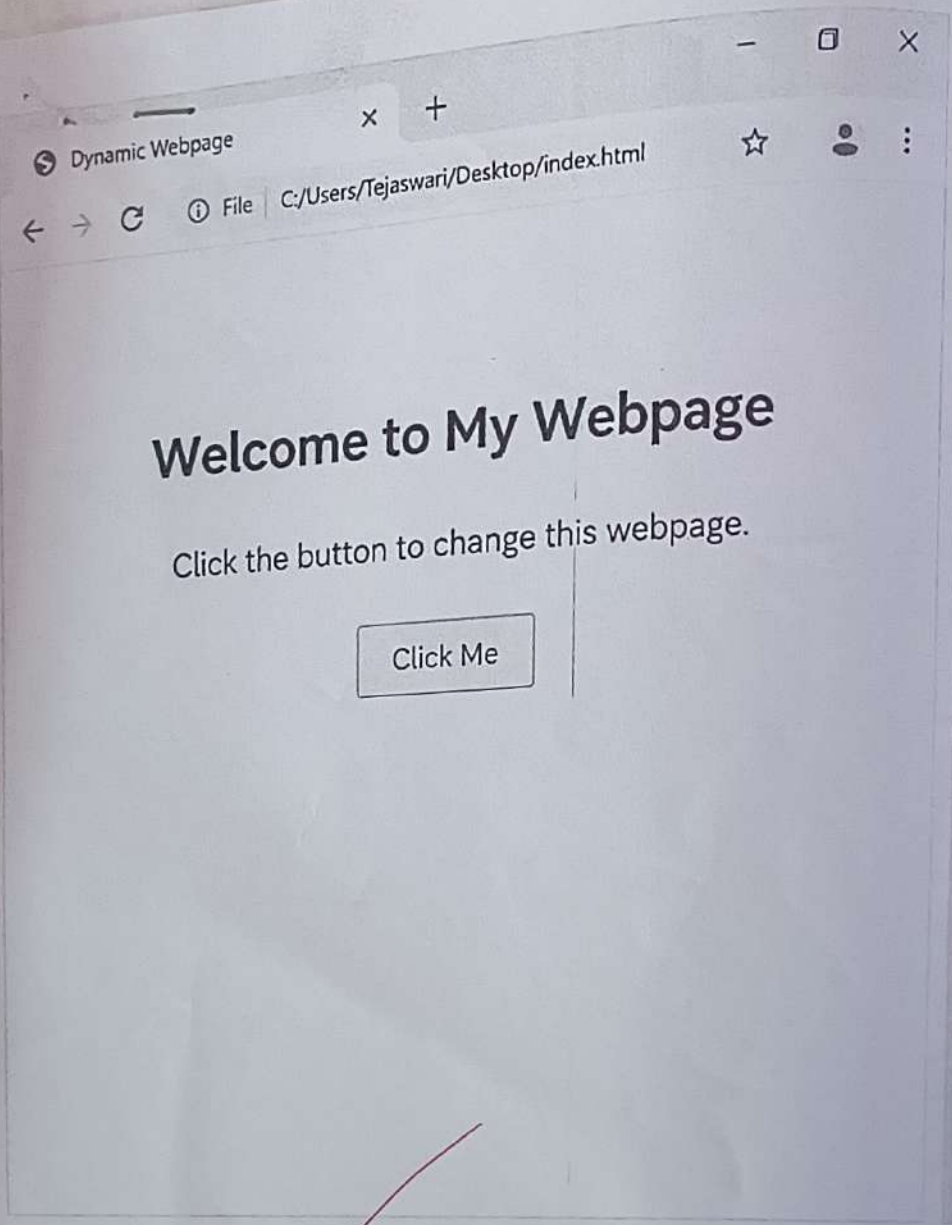
Here are some of the most Commonly used ones;

1) fs = file system operations
2) https - HTTP Server and client
3) Path - File Path utilities
4) os - operating system utilities
5) events - Event handling
6) Util - Utility functions
7) Stream - Stream handling
8) Crypto - Cryptographic functions
9) Url - URL parsing
10) querystring - URL query string handling

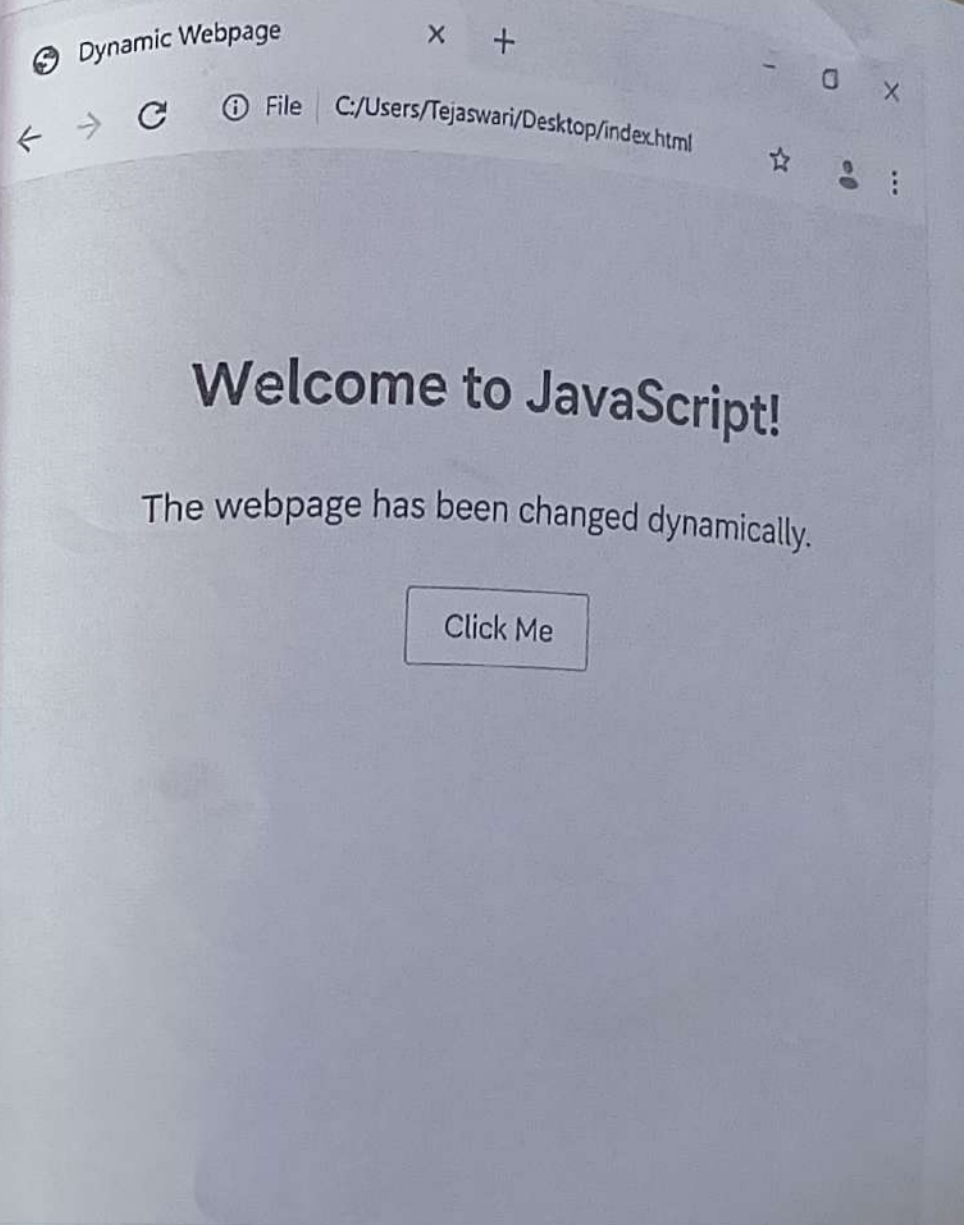
Example:- Simple HTTP Server

```
http.CreateServer((req, res) => {
    res.writeHead(200, {'Content-Type':
                        'text/html'});
    res.end('Hello World!');
}).listen(8080);
```

O/P :- Hello World!



Before clicking the button



After clicking the button

Experiment-06

06) Develop mini application using express and NodeJs

* Simple mini application using nodeJs + Express.js

Student Management System

1) Add a Student
2) View all students
3) delete a student
4) User Express.js routes
5) Simple HTML interface

Project Structure:

```
Student.app/
├── Server.js
└── public
    └── index.html
```

Server.js:

```
Const express = require("express");
Const app = express();
Const PORT = 3000;
let students = [];
app.use(express.json());
app.use(express.static("Public"));
app.get("/students", (req, res) => {
    res.json(students);
});
```

```
//Add
app.post("/students", (req, res) => {
    Const student = {
        id : students.length+1,
        name: req.body.name,
        Course: req.body.Course
    };
    Students.push(student);
    res.json({message: "student added succesfully",
        student : student });
});
//delete
app.delete("/students/:id", (req,res) => {
    Const id= parseInt (req.parame id);
    Students = students.filter(student => student.id!
                                    == id);
    res.json({
        message: "student deleted successfully"});
});
// Start Server
app.listen(PORT, () => {
    Console.log('server running at http://localhost:
                ${PORT}');
});
```

Public/index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title> Student Management </title>
</head>
<body>
    <h1> Student Management System </h1>
    <input type = "text" id = "name"
    placeholder = "Enter Student name" >
    <input type = "text" id = "Course"
    placeholder = "Enter Course">
    <button onclick = "addStudent()" > Add Student
                                    </button>
    <h2> Student List </h2>
    <ul id : "Studentlist" ></ul>
    <script>
        function addStudent(){
            let name = document.getElementById("name").
                                    value;
            let Course = document.getElementById("Course").
                                    value;
            fetch ("/students", {
                method = "Post",
                headers = {
                    "Content-Type": "application/json"},
```

```
body: JSON.stringify({
        name: name,
        Course: Course
    })
})
.then(response => response.json())
.then(data => {
        alert(data.message);
document.getElementById("name").value = "";
document.getElementById("course").value = "";
displayStudents();
});
}

function displayStudents(){
    fetch("/students").then(response => response.json())
    .then(students => {
        let list = document.getElementById("studentList");
        let.innerHTML = "";
    students.forEach(student => {let li =
            document.createElement("li");
            li.innerHTML = student.id + "-" +
                           student.name + "-" +
                           student.course + "<button
    onclick='deleteStudent(" + student.id + ")'>Delete
    </button>";
    list.appendChild(li);
        });
    });
```

```
function deleteStudent(id){
    fetch("/students/"+id,{method:"DELETE"})
.then(response => response.json())
.then(data => {
            alert(data.message);
            displayStudents();
    });
}
displayStudents();
</script>
</body>
</html>
```

O/P

Student Management System

Enter Student Name

Enter Course

Add Student

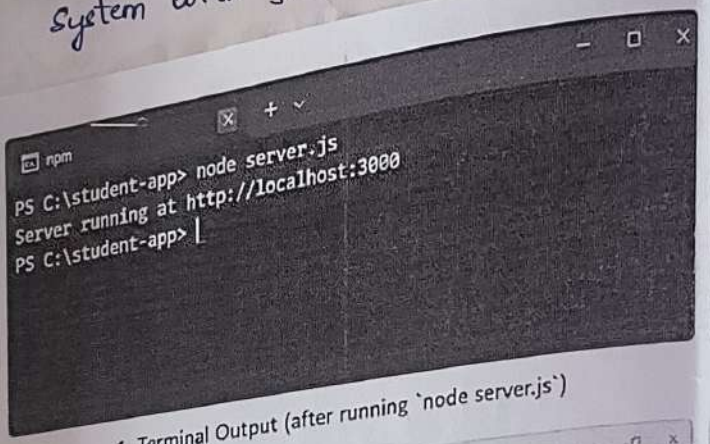
Student List

1) Lakshitha . CSM delete

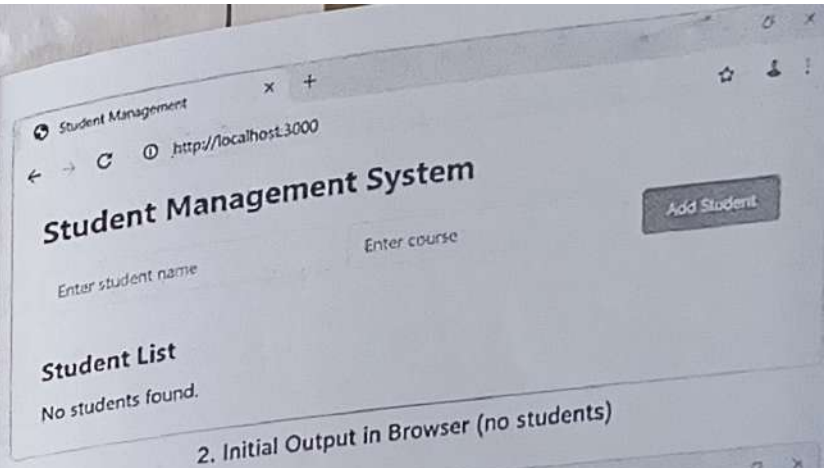
2) Lucky . CSE delete

* The Express.js server is succesfully started at http://localhost:3000

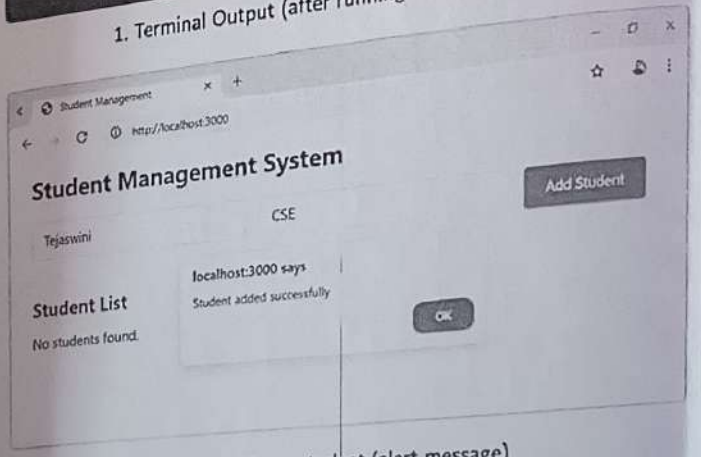
The browser displays the
System with fields



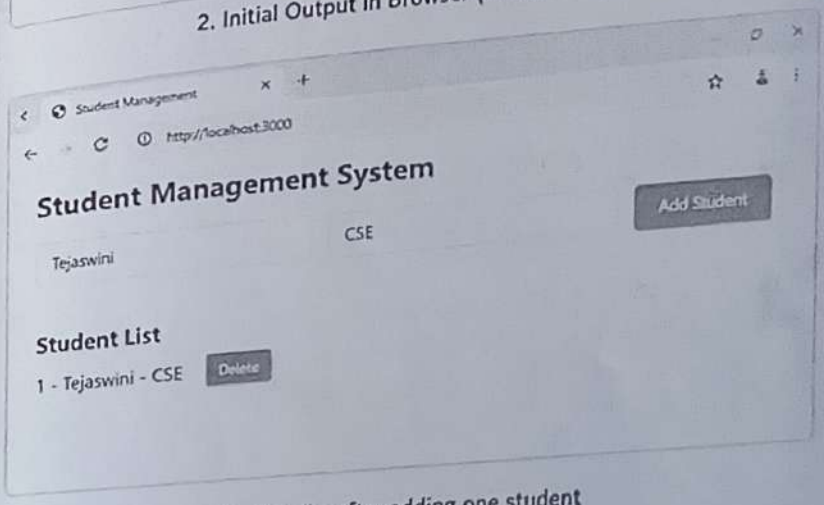
1. Terminal Output (after running `node server.js`)



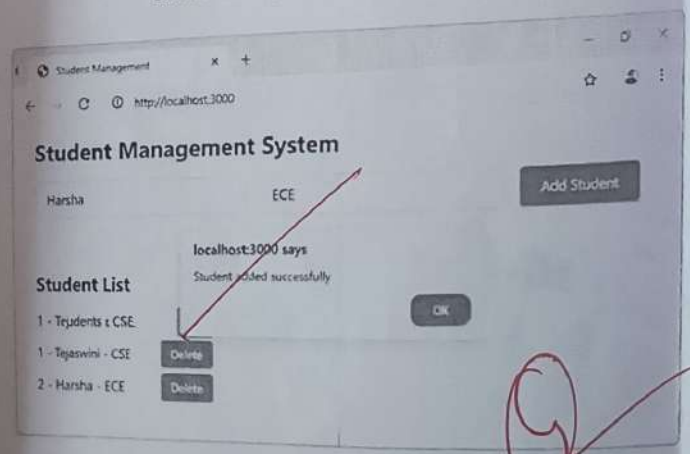
2. Initial Output in Browser (no students)



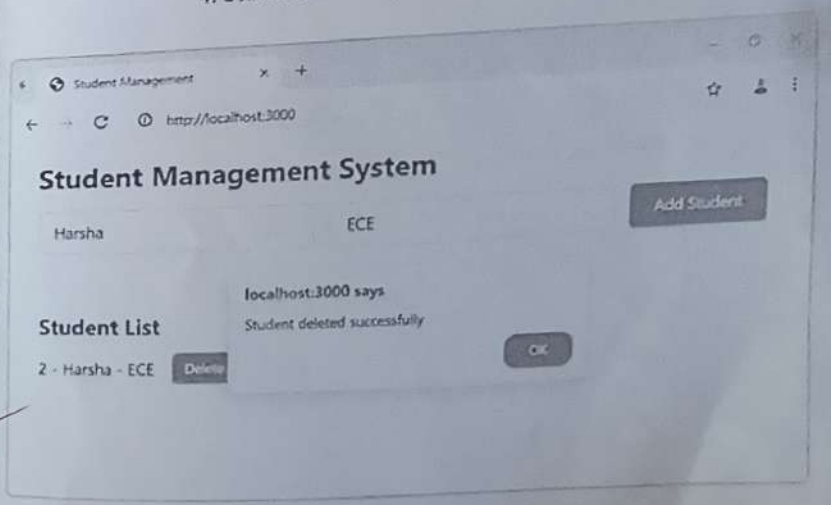
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment-07

7) Create a Functional Component and class component using react

Aim:- To develop a react application using Functional Components and class components to display student details, manage state, handle button clicks and dynamic-ally update data on a web page

Functional Component:-

A Functional Component is a javascript function that returns JSX to display the user interface. It uses React Hooks such as usestate to manage state and update the Webpage dynamically.

Program

```
import React, {useState} from 'react';
function studentFunctional(){
    Const [name, setName] = useState("Lakshitha")
    Const [course, setCourse] = UseState("CSM");
    const [count, setCount] = useState(0);
    Const changeName = () => {
        setName("Lucky");
    };
    Const changeCourse = () => {
        setCourse("CSD");
    };
    Const incrementCount = () => {
        setCount(count+1);
    };
```

```
return (
    <div style={{
        border: "2px solid blue",
        Padding: "20 px",
        margin: "20px",
        borderRadius: "10px",
        textAlign: "Center"
    }}>
    <h1> StudentDetails </h1>
    <h2> Function Component </h2>
    <p><b>Name:</b> {name} </p>
    <p><b>Course:</b> {Course} </p>
    <p><b> Button clicks: </b> {Count} </p>
    <button onClick={changeName}> changeName </button>
    <button onClick={changeCourse} > changeCourse
                                    </button>
    <button onClick={incrementCount}> Increment
                                    </button>
    </div>
);
```

Output:

When the user clicks the buttons, the name, course and count change dynamically without reloading the Webpage.

- **Class Component:-**

A class Component is an ES6 Javascript class that extends React.Component. It uses the render() method to return JSX and manages state using this.state and this.setState()

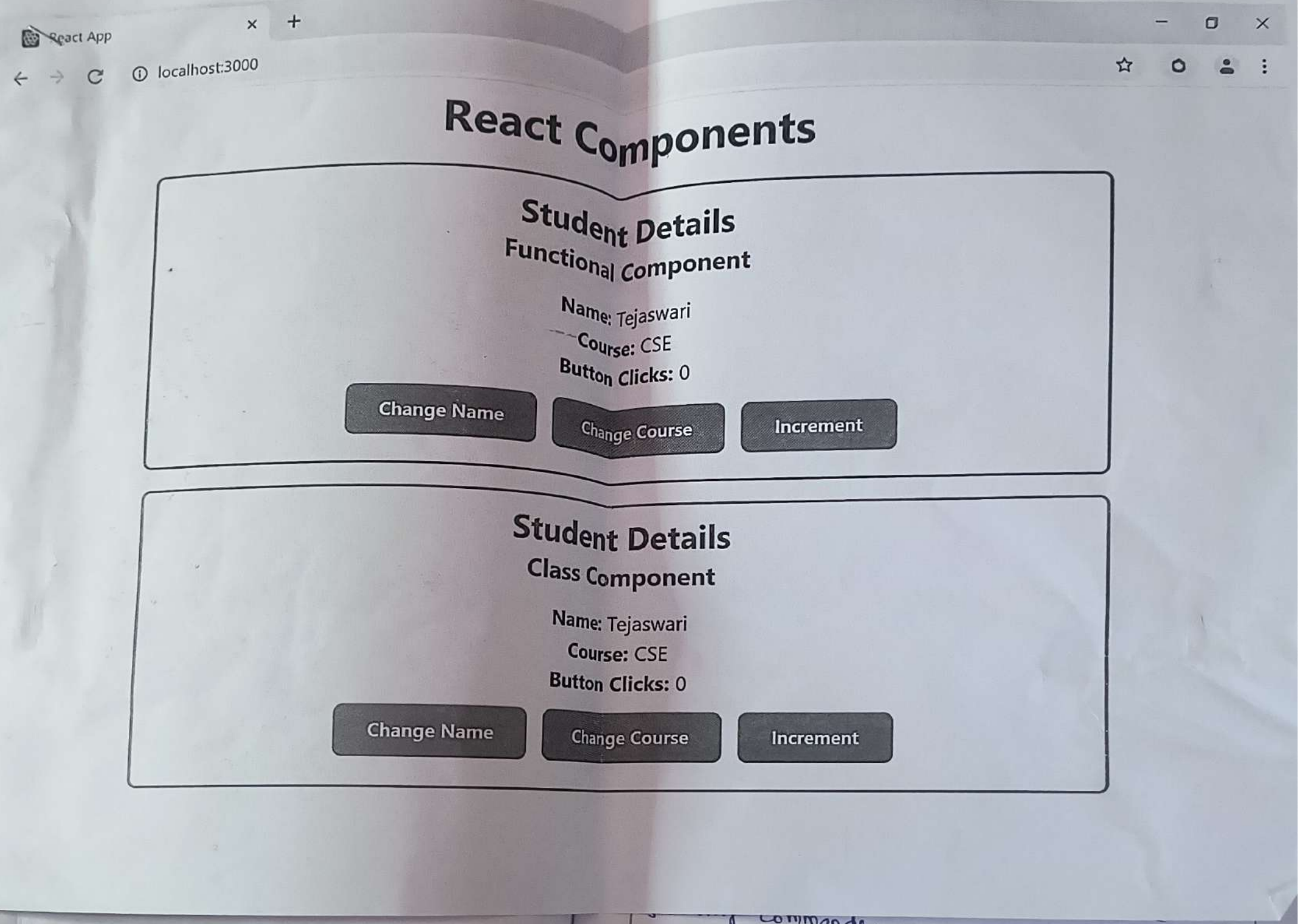
Program:-

```
import React, {Component} from 'react';
class StudentClass extends Component {
    Constructor(props) {
        super(props);
        this.state = {
            name: "Lakshitha",
            Course: "Computer Science",
            Count: 0
        };
    }
    changeName = () => {
        this.setState({
            name: "Teju"
        });
    };
    ChangeCourse = () => {
        this.setState({
            Course: "CSM"
        });
    };
    incretment Count = () => {
        this.setState(prevState) => ({
            Count = prevstate.Count + 1
        }));
    };
```

```
render(){
    return(
        <div style={{
                border: "2px solid green",
                Padding: "20px",
                margin: "20px",
                borderRadius: "10px",
                textAlign: "Center"
        }}>
        <h1> Student Details </h1>
        <h2> Class Component </h2>
    <p><b> Name: </b> {this.state.name} </p>
    <p> <b> Course: </b> {this.state.course} </p>
    <button onClick={this.changeName}>
                changeName </button>

    <button onClick={this.changeCourse}>
                Change Course </button>

    <button onclick={this.incrementCount}>
                Increment </button>
</div>
```


React App
localhost:3000
React Components
Student Details
Functional Component
Name: Tejaswari
Course: CSE
Button Clicks: 0
Change Name
Change Course
Increment
Student Details
Class Component
Name: Tejaswari
Course: CSE
Button Clicks: 0
Change Name
Change Course
Increment

Experiment-08

8) Implement http methods using JSON Server

Aim:- To implement HTTP methods such as GET, POST, PUT, PATCH, and DELETE using JSON Server to Perform CRUD (Create, Read, Update and Delete) operations on student records.

Theory:- JSON Server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch.

HTTP methods are used to Communicate between a client and a Server

* GET - Retrieves data from the server
* POST - Adds new data to the server
* PUT - Replaces an existing record
* PATCH - Updates selected fields of a record
* DELETE - Removes a record from the Server

a) Create a JSON Server:

Step-1: Initialize the project

Open the VS Code Terminal and execute the following Commands:

```
mkdir JSONServer
cd. JSONServer
npm init -y
npm install json server.
```

Step-2:- Create db-json

Create a file named db.json and add the following student data

```
{
  "Students": [
    { "id": "1",
      "name": "Lakshitha",
      "Course": "csm"
      "marks": 90
    },
    {
      "id": "2"
      "name": "Chaitu",
      "Course": "csm",
      "marks": 90
    },
    {
      "id": "3",
      "name": "Pavani",
      "Course": "IT",
      "marks": 88
    }
```

Step-3:- Start the JSON Server

For the Current JSON Server Version, execute:

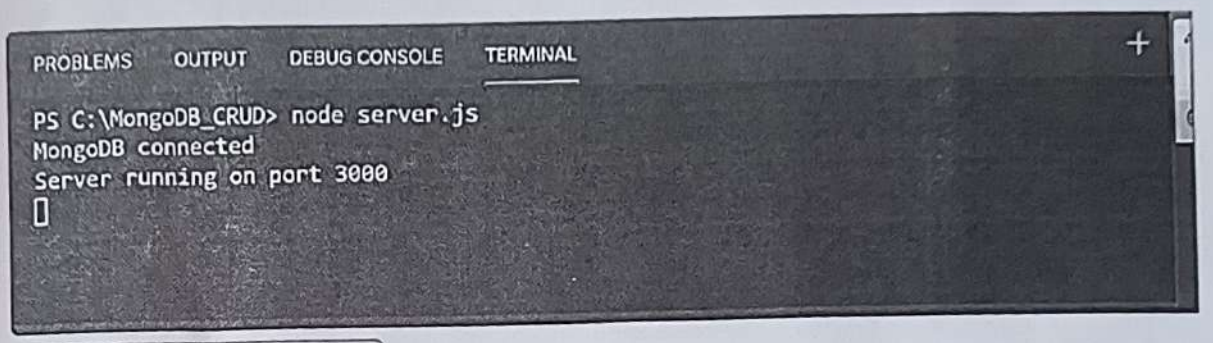
npm json.server db.json

The server start at:

http://localhost:3000

The Student records can be accessed using

http://localhost:3000/students



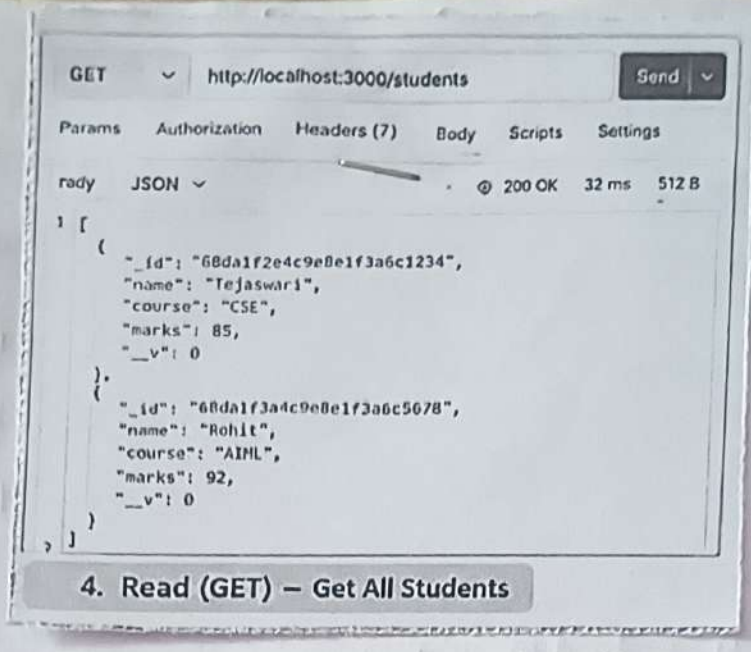
1. Terminal Output

2. Implementation of HTTP methods:-

a) GET Method: The GET method is used to retrieve student records from JSON server

Code:

```
fetch('http://localhost:3000/students')
.then(response => response.json())
.then(data => {
        Console.log(data);
})
.Catch(error => {
        Console.log(error);
});
```



4. Read (GET) – Get All Students

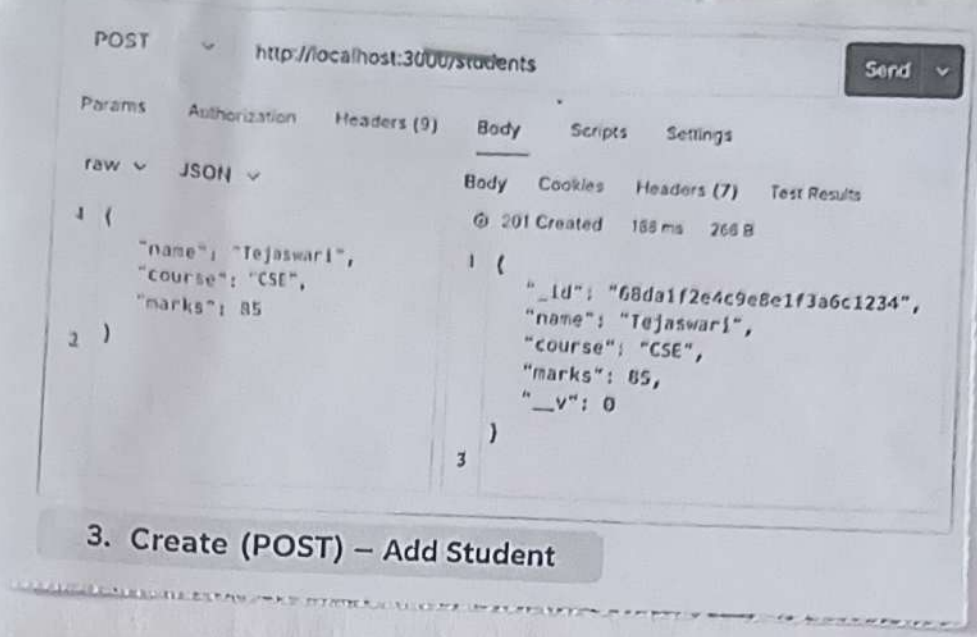
B) POST method:- The POST method is used to insert a new student record into the server.

Code:-

```
fetch ('http://localhost:3000/students', {
    method : 'POST',
    headers : {
        'Content-Type' : 'application/json'
    },
    body : JSON.stringify ({
        name : 'Rahul',
        Cource : 'CSE',
        marks : 92
    })
})
.then (response => response.json())
.then (data => {
        Console.log (data);
})
.catch (error => {
        Console.Log (error);
```

});

o/p



c) PUT Method :- The PUT method is used to replace an existing student record with updated data.

Code : fetch ('http://localhost:3000/students/1', {

method : 'PUT',

headers: {

'Content-Type': 'application/json'

},

body : JSON.stringify ({

name : 'Lakshitha',

Course : 'AIML',

marks: 95

})

})

.then(response => response.json())

.then(data => {

Console.log(data);

})

.catch (error => {

Console.Log (error);

});

O/P

PUT http://localhost:3000/students/68da1f2e4c9e8e1f3a6c1234 Send

Params Authorization Headers (9) Body Scripts Settings

raw JSON

```
{
    "name": "Tejaswari",
    "course": "AIML",
    "marks": 95
}
```

Body Cookies Headers (7) Test Results

200 OK 24 ms 268 B

```
{
    "_id": "68da1f2e4c9e8e1f3a6c1234",
    "coure": "Tejaswari",
    "course": "AIML",
    "marks": 95,
    "__v": 0
}
```

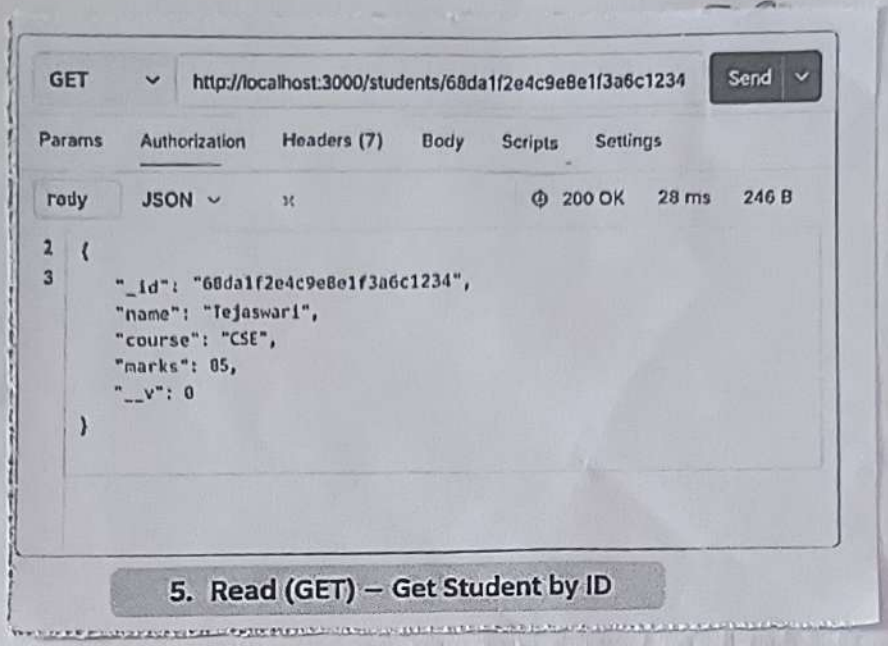
6. Update (PUT) – Update Student

D) PATCH Method:- The PATCH method is used to updat only selected fields of an existing student record.

Code:

```
fetch ('http://localhost:3000/students/2', {
    method : 'PATCH',
    headers : {
        'Content-Type' : 'application/json'
    },
    body : JSON.stringify ({
        marks : 96
    })
})
.then (response => response.json())
.then (data => {
    Console.log (data);
})
.catch (error => {
    Console.log (error);
});
```

O/P



E) DELETE method: The DELETE method is used to remove an existing Student record from the server

Code: fetch ('http://localhost:3000/students/3', {

method: 'DELETE'

})

.then (response => {

if (response.OK) {

Console.log("student deleted Successfully");

}

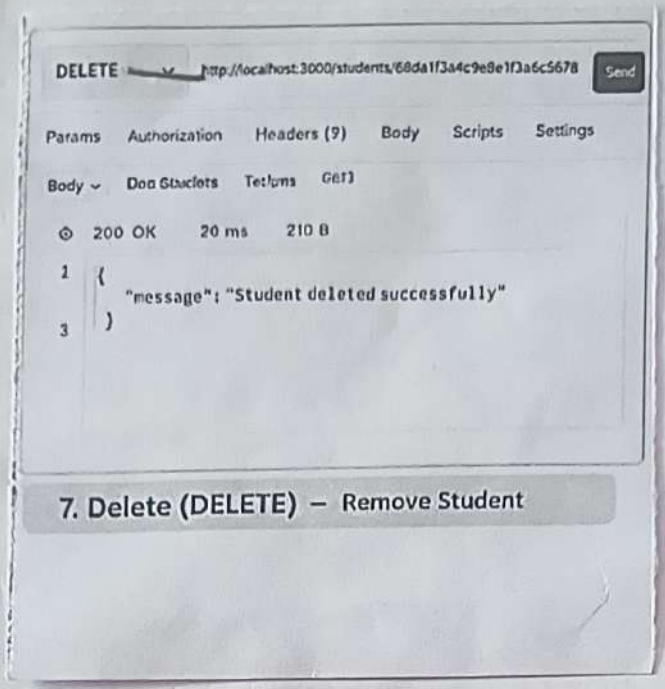
})

.catch(error => {

Console.log(error);

});

O/P



* After executing the POST, PUT, PATCH and DELETE operations in sequence, the remaining student will be:

O/P :

Thus, HTTP methods GET, POST, PUT, PATCH and DELETE were successfully implemented using JSON Server. The CRUD operations were performed on student records, the data was retrieved, inserted, updated and deleted using HTTP requests.

Experiment-09

9) Develop CRUD operations using MongoDB

Aim:- To develop a webapplication using Node.js, Express.js and MongoDB. to perform CRUD operations (Create, read, update, delete) on student records.

Theorey:- MongoDB is a NoSQL database that stores data in flexible, ~~Tables~~ JSON like documents, CRUD operations are used to manage data in a database

Mongoose in an object Data Modeling (ODM) library for MongoDB and Node.js. It helps define schemas, interact with collections and Perform database operations

1) Installation and Project Setup:-

```
mkdir MongoDB-CRUD
cd MongoDB-CRUD
npm init -y
npm install express mongoose
```

2) Program: Server.js

```
Const express = require('express');
Const mongoose = require("mongoose");
Const app = express();
Const PORT = 3000;
```

```
app.use(express.json());
mongoose.connect('mongodb://127.0.0.1:27017
                  /College')
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));

//Create student
Const studentSchema = new
    mongoose.schema({
        name: {
            type: String,
            required: true
        },
        Course: {
            type: String,
            required: true
        },
        marks: {
            type: Number,
            required: true
        }
});

//Create student model
Const student =
    mongoose.model('student', studentSchema);
```

```
//Create : Add
app.post('/students', async(req,res) =>{
    try {
        Const student = new Student(req.body);
        Const savedStudent = await Student.
                                        save();
    res.status(201).json(saved student);
    } Catch(error){
        res.status(400).json({message:
                    error.message});
        }
    });
//Read : Get all students
app.get('/students', async(req,res) => {
    try {
        Const students = await
        Student.find();
        res.status(500).json({message: error.
                                        message});
        }
    });
//READ: Get student by ID
app.get('/students/:id', async(req,res) => {
    try {
        Const student = await
```

```
Student.findById(req.params.id);
if (!student){
    return res.status(404).json({
        message: "student not found"
    });
}
res.status(200).json(student);
} catch(error){
    res.status(400).json({message: error.message});
}
});

//UPDATE
app.put('/students/:id', async(req,res)=>{
try{
    const student = await
    Student.findByIdAndUpdate(
        req.params.id,
        req.body
        {new: true, runValidators: true}
    );
if (!student){
    return res.status(404).json({
        message: "student not found"
    });
}
```

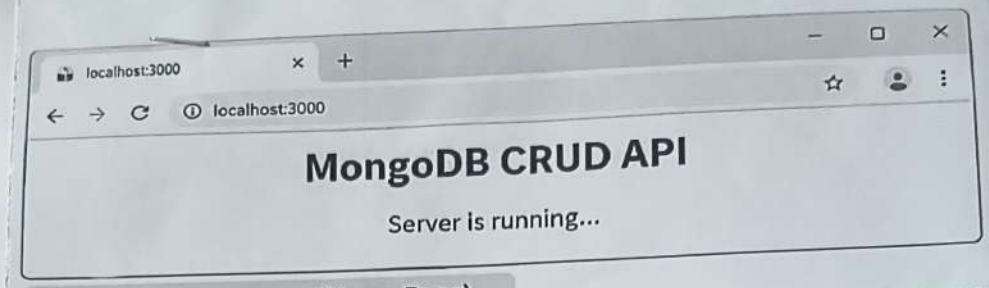
```
    res.status(200).json(studen
} catch(error){
    res.status(400).json({m
}
});

//DELETE
app.delete('/students/:id
try{
    const student = awa
        findBy
    );
    if (!student){
        return res.
            messag
        });
    }
    res.status(200).
        message: "s
    });
} catch(error){
    res.status(40
}
});
```

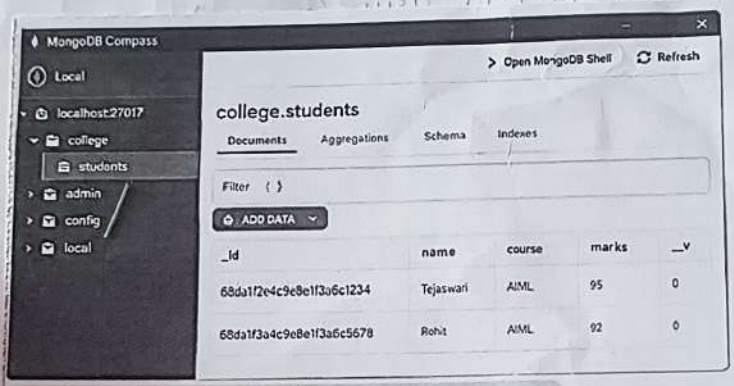
```
    res.status(200).json(student);
} catch (error) {
    res.status(400).json({message: error.message});
  }
};

//DELETE
app.delete('/students/:id', async(req,res) => {
  try{
    Const student = await Student.
                findByIdAndDelete(
                    req.params.id
    );
    if (!student){
        return res.status(404).json({
            message: "student not found"
        });
    }
    res.status(200).json({
        message: "student deleted succesfully"
    });
} catch (error){
    res.status(400).json({message: error.message});
  }
});
```





2. Chrome Browser (Home Page)



8. MongoDB Compass – Database View